

NAVIGATION STACK

1. INSTALL NAV2 AND DEPENDENCIES

What you're doing:

You're installing the **Nav2 navigation software**, which gives your catamaran the ability to map, localize, and plan paths.

Command:

```
sudo apt install ros-<ros2-humble>-navigation2 ros-<ros2-humble>-nav2-bringup
```

2. SOURCE ROS 2 AND SET UP YOUR WORKSPACE

What you're doing:

You're preparing your computer to use ROS 2 and making a workspace to keep all your robot's files organized.

Command:

```
source /opt/ros/<ros2-humble>/setup.bash
mkdir -p ~/catamaran_ws/src
cd ~/catamaran_ws
colcon build
source install/setup.bash
```

This ensures all ROS 2 commands and packages work in your terminal.

3. CREATE OR USE A MAP (SLAM OR PRE-MADE)

What you're doing:

You're giving your catamaran a map of its environment, so it knows where it is and where it can go.

If you want the catamaran to build its own map (SLAM):

```
ros2 launch slam_toolbox online_async_launch.py
```

- Drive the catamaran around (using keyboard or joystick teleop) to let it scan the area.
- **Save the map:**

```
ros2 run nav2_map_server map_saver_cli -f ~/catamaran_ws/maps/my_map
```

Now you have a map file your navigation stack can use.

4. PREPARE NAV2 CONFIGURATION FILES

What you're doing:

You're telling Nav2 about your robot's size, sensors, and how it should plan and control its movement.

- Edit or create **nav2_params.yaml** in your **~/catamaran_ws/src/your_catamaran_navigation/config/** directory.
- Set values for your robot's footprint, sensor topics, velocity limits, etc.

5. LAUNCH THE NAVIGATION STACK

What you're doing:

You're starting all the Nav2 programs that handle localization, path planning, and movement.

Command:

```
ros2 launch nav2_bringup navigation_launch.py map:=/absolute/path/to/my_map.yaml  
params_file:=/absolute/path/to/nav2_params.yaml
```

This starts the navigation system with your map and configuration.

6. SET THE INITIAL POSE IN RVIZ2

What you're doing:

You're telling the system exactly where the catamaran is on the map, so it can localize itself.

Command:

```
rviz2
```

- In RViz2, use the “2D Pose Estimate” tool to click the catamaran’s starting location on the map.

7. SEND NAVIGATION GOALS

What you're doing:

You're telling the catamaran where you want it to go.

- In RViz2, use the “Nav2 Goal” tool to click a destination on the map.
- Or, use a Python script to send goals programmatically.

8. WATCH, TEST, AND TUNE

What you're doing:

You're observing the catamaran's behavior, checking logs, and adjusting parameters for better performance.

- Watch the catamaran move in RViz2.
- Adjust `nav2_params.yaml` as needed for things like speed, obstacle avoidance, etc.
- Restart the navigation stack after changes.

SUMMARY TABLE

Step	What You Do	Command/Action
1	Install Nav2	<code>sudo apt install ...</code>
2	Set up workspace	<code>mkdir -p ~/catamaran_ws/src ...</code>
3	Make or use a map	<code>ros2 launch slam_toolbox ...</code> / use existing map
4	Prepare Nav2 config	Edit <code>nav2_params.yaml</code>
5	Launch navigation stack	<code>ros2 launch nav2_bringup ...</code>
6	Set initial pose in RViz2	Open RViz2, use "2D Pose Estimate"
7	Send navigation goals	Use RViz2 "Nav2 Goal" or a Python script
8	Watch, test, and tune	Observe, edit params, re-launch as needed

WORKING WITH NAV2

1. File Checklist for Nav2

You'll need these files (as described earlier):

```
catamaran_navigation/  
├── config/  
│   ├── nav2_params.yaml  
│   ├── slam_toolbox_params.yaml  
│   └── behavior_trees/  
│       ├── navigate_to_pose_bt.xml  
│       └── navigate_through_poses_bt.xml  
├── launch/  
│   ├── slam.launch.py  
│   └── navigation.launch.py  
├── maps/  
│   ├── my_map.yaml  
│   └── my_map.pgm  
└── package.xml
```

2. Step-by-Step: Mapping and Navigation

A. Configure SLAM Toolbox

File: `config/slam_toolbox_params.yaml`

Purpose: Sets LIDAR topic, map resolution, etc.

A. Configure SLAM Toolbox

1. Purpose

- **SLAM Toolbox** enables your robot to **build a map** of the environment in real time using LIDAR data, while also tracking its position on that map (simultaneous localization and mapping).
- The configuration file (`slam_toolbox_params.yaml`) sets up which topics to use, the robot's reference frames, map resolution, and other performance-related parameters.

`config/slam_toolbox_params.yaml`

Below is a minimal but practical configuration for your catamaran:

```
slam_toolbox:  
  ros__parameters:  
    use_sim_time: false           # Set to true if using simulation or  
rosbag  
    odom_frame: odom  
    map_frame: map  
    base_frame: base_link  
    scan_topic: /scan            # Make sure this matches your LIDAR topic  
    resolution: 0.05              # 5cm per grid cell  
    max_laser_range: 12.0         # Set to your LIDAR's max range  
    minimum_travel_distance: 0.2  
    minimum_travel_heading: 0.174
```

```

scan_buffer_size: 10
scan_buffer_maximum_scan_distance: 10.0
do_loop_closing: true
loop_match_minimum_chain_size: 10
loop_match_maximum_variance_coarse: 3.0
loop_match_maximum_variance_fine: 0.5
loop_search_maximum_distance: 3.0
mode: mapping # Use 'mapping' for building the map

```

- **odom_frame, map_frame, base_frame:** Should match your URDF and TF tree.
- **scan_topic:** Must match your LIDAR's ROS topic (commonly /scan).
- **resolution:** Smaller values = higher map detail, but more CPU/memory use.
- **mode: mapping:** Enables map building (use localization later for navigation only).

3. Where to Put the File

Save this file as:

catamaran_navigation/config/slam_toolbox_params.yaml

4. Launching SLAM Toolbox

launch file: launch/slam.launch.py

```

from launch import LaunchDescription
from launch_ros.actions import Node
import os

def generate_launch_description():
    # Update these paths for your project directory structure
    urdf_file = os.path.join(
        os.getenv('HOME'),
        'catamaran_ws',
        'src',
        'catamaran_description',
        'urdf',
        'catamaran.urdf.xacro'
    )
    slam_params_file = os.path.join(
        os.getenv('HOME'),
        'catamaran_ws',
        'src',
        'catamaran_navigation',
        'config',
        'slam_toolbox_params.yaml'
    )

    return LaunchDescription([
        Node(

```

```

        package='robot_state_publisher',
        executable='robot_state_publisher',
        arguments=[urdf_file],
        output='screen'
    ),
    Node(
        package='slam_toolbox',
        executable='async_slam_toolbox_node',
        name='slam_toolbox',
        output='screen',
        parameters=[slam_params_file]
    ),
    # Add your sensor driver nodes here if needed
])

```

To launch:

`ros2 launch catamaran_navigation slam.launch.py`

5. Verifying Everything Works

- **Check that /scan is publishing:**

bash

`ros2 topic list`

`ros2 topic echo /scan --once`

- **SLAM Toolbox should publish /map** (the occupancy grid).
- **Visualize in RViz:**

Add /map, /scan, /tf topics to see mapping in real time.

6. Saving the Map

Once you've mapped the area:

`ros2 run nav2_map_server map_saver_cli -f ~/catamaran_navigation/maps/my_map`

This will generate `my_map.yaml` and `my_map.pgm` for use in navigation mode.

Summary Table

Parameter	Example Value	Purpose
scan_topic	/scan	LIDAR topic for SLAM
base_frame	base_link	Robot's main body frame
odom_frame	odom	Odometry frame
map_frame	map	Map frame
resolution	0.05	Map cell size (meters)
max_laser_range	12.0	Max LIDAR range (meters)
mode	mapping	SLAM mode for building map

In summary:

- Edit and save your `slam_toolbox_params.yaml` as above.
- Make sure your LIDAR is publishing to /scan.
- Launch SLAM Toolbox with your launch file.

- Visualize mapping in RViz.

display.launch.py

```
from launch import LaunchDescription
from launch_ros.actions import Node
from launch.actions import ExecuteProcess
import os

def generate_launch_description():
    # Get the path to the URDF/Xacro file
    pkg_share = os.path.join(
        os.path.expanduser('~'),
        'catamaran_ws', 'src', 'catamaran_description'
    )
    urdf_file = os.path.join(pkg_share, 'urdf', 'catamaran.urdf.xacro')

    return LaunchDescription([
        # Joint State Publisher GUI (for interactive joint control, optional for
        # fixed robots)
        Node(
            package='joint_state_publisher_gui',
            executable='joint_state_publisher_gui',
            name='joint_state_publisher_gui',
            output='screen'
        ),
        # Robot State Publisher (publishes TF and robot_description)
        Node(
            package='robot_state_publisher',
            executable='robot_state_publisher',
            name='robot_state_publisher',
            output='screen',
            arguments=[urdf_file]
        ),
        # Optionally, launch RViz2 with a blank config
        ExecuteProcess(
            cmd=['rviz2', '-d', ''],
            output='screen'
        )
    ])
```

How to Use

Place this file at:

`catamaran_ws/src/catamaran_description/launch/display.launch.py`

Build your workspace:

`cd ~/catamaran_ws`

`colcon build`

`source install/setup.bash`

Launch the model:

```
ros2 launch catamaran_description display.launch.py
```

In RViz2:

Set "Fixed Frame" to base_link (or your URDF root link)

Add "RobotModel" and "TF" displays to see your catamaran and all frames

- Save your map for navigation use.

NAVIGATION AFTER MAPPING

1. Prepare Your Navigation Parameters

File: `catamaran_navigation/config/nav2_params.yaml`

Purpose: This YAML file configures all Nav2 components—controllers, planners, costmaps, behavior trees, etc. It is the heart of your navigation configuration.

Key things to set:

- **Frames:** map, odom, base_link (should match your URDF and SLAM config)
- **Velocity limits:** Based on your catamaran's capabilities
- **Footprint:** Accurate shape/size of your robot
- **Sensor topics:** LIDAR, ultrasonics, GNSS, etc.
- **Plugins:** For planners and controllers

snippet:

```
amcl:
  ros__parameters:
    use_sim_time: false
    base_frame_id: base_link
    odom_frame_id: odom
    scan_topic: /scan
    min_particles: 500
    max_particles: 2000
    alpha1: 0.2
    alpha2: 0.2
    alpha3: 0.2
    alpha4: 0.2
    alpha5: 0.2
    laser_max_beams: 60

map_server:
  ros__parameters:
    use_sim_time: false
    yaml_filename:
/home/$USER/catamaran_ws/src/catamaran_navigation/maps/my_map.yaml

bt_navigator:
  ros__parameters:
    use_sim_time: false
    global_frame: map
    robot_base_frame: base_link
    odom_topic: /odom
    bt_loop_duration: 10
    default_server_timeout: 20
    default_nav_to_pose_bt_xml:
/home/$USER/catamaran_ws/src/catamaran_navigation/config/behavior_trees/navigate_
to_pose_bt.xml
```



```

    default_nav_through_poses_bt_xml:
/home/$USER/catamaran_ws/src/catamaran_navigation/config/behavior_trees/navigate_
through_poses_bt.xml

planner_server:
  ros__parameters:
    use_sim_time: false
    expected_planner_frequency: 1.0
    planner_plugins: ["GridBased"]
    GridBased:
      plugin: "nav2_navfn_planner/NavfnPlugin"
      tolerance: 0.5
      use_astar: false
      allow_unknown: true

controller_server:
  ros__parameters:
    use_sim_time: false
    controller_frequency: 10.0
    controller_plugins: ["FollowPath"]
    FollowPath:
      plugin: "dwb_core::DWBLocalPlanner"
      min_vel_x: 0.2
      max_vel_x: 1.2
      max_vel_theta: 1.0
      min_vel_theta: -1.0
      acc_lim_x: 1.0
      acc_lim_theta: 1.0
      decel_lim_x: -1.0
      decel_lim_theta: -1.0
      vx_samples: 20
      vy_samples: 5
      vtheta_samples: 20
      sim_time: 2.0
      linear_granularity: 0.05
      angular_granularity: 0.025
      xy_goal_tolerance: 0.25
      yaw_goal_tolerance: 0.25
      trans_stopped_velocity: 0.25
      short_circuit_trajectory_evaluation: true
      stateful: true

local_costmap:
  ros__parameters:
    use_sim_time: false

```

```

global_frame: odom
robot_base_frame: base_link
update_frequency: 5.0
publish_frequency: 2.0
rolling_window: true
width: 5.0
height: 5.0
resolution: 0.05
footprint: [[-1.175, -0.485], [-1.175, 0.485], [1.175, 0.485], [1.175, -
0.485]]
plugins: ["obstacle_layer", "inflation_layer"]
obstacle_layer:
  enabled: true
  observation_sources: scan, ultrasonic_front, ultrasonic_back
  scan:
    topic: /scan
    sensor_frame: rplidar_link
    data_type: LaserScan
    marking: true
    clearing: true
  ultrasonic_front:
    topic: /ultrasonic_front
    sensor_frame: ultrasonic_front_link
    data_type: Range
    marking: true
    clearing: true
  ultrasonic_back:
    topic: /ultrasonic_back
    sensor_frame: ultrasonic_back_link
    data_type: Range
    marking: true
    clearing: true
  inflation_layer:
    enabled: true
    inflation_radius: 0.6

global_costmap:
  ros__parameters:
    use_sim_time: false
    global_frame: map
    robot_base_frame: base_link
    update_frequency: 1.0
    publish_frequency: 1.0
    width: 50.0
    height: 50.0

```

```

resolution: 0.05
footprint: [[-1.175, -0.485], [-1.175, 0.485], [1.175, 0.485], [1.175, -
0.485]]
plugins: ["static_layer", "obstacle_layer", "inflation_layer"]
static_layer:
  enabled: true
obstacle_layer:
  enabled: true
  observation_sources: scan, ultrasonic_front, ultrasonic_back
scan:
  topic: /scan
  sensor_frame: rplidar_link
  data_type: LaserScan
  marking: true
  clearing: true
ultrasonic_front:
  topic: /ultrasonic_front
  sensor_frame: ultrasonic_front_link
  data_type: Range
  marking: true
  clearing: true
ultrasonic_back:
  topic: /ultrasonic_back
  sensor_frame: ultrasonic_back_link
  data_type: Range
  marking: true
  clearing: true
inflation_layer:
  enabled: true
  inflation_radius: 0.6

recoveries_server:
  ros__parameters:
    use_sim_time: false

waypoint_follower:
  ros__parameters:
    use_sim_time: false
    loop_rate: 20
    stop_on_failure: false

```

2. Create a Navigation Launch File

File: `catamaran_navigation/launch/navigation.launch.py`

Purpose: Launches Nav2, robot_state_publisher, and any sensor drivers needed for navigation.
`navigation.launch.py`

```

from launch import LaunchDescription
from launch_ros.actions import Node
import os

def generate_launch_description():
    pkg_dir = os.path.expanduser('~/.catamaran_ws/src/catamaran_navigation')
    urdf_file =
os.path.expanduser('~/.catamaran_ws/src/catamaran_description/urdf/catamaran.urdf.
xacro')
    map_file = os.path.join(pkg_dir, 'maps', 'my_map.yaml')
    nav2_params = os.path.join(pkg_dir, 'config', 'nav2_params.yaml')

    return LaunchDescription([
        Node(
            package='robot_state_publisher',
            executable='robot_state_publisher',
            arguments=[urdf_file],
            output='screen'
        ),
        Node(
            package='nav2_bringup',
            executable='bringup_launch.py',
            output='screen',
            parameters=[nav2_params],
            arguments=[
                f'map:={map_file}',
                'use_sim_time:=false'
            ]
        ),
        # Add sensor nodes here if not launched elsewhere
    ])

```

Adjust the file paths to match your workspace.

3. Launch the Navigation Stack

Command:

```
ros2 launch catamaran_navigation navigation.launch.py
```

- This will start Nav2, load your map, and bring up all navigation nodes.

4. Use RViz for Navigation

- Open RViz:

RViz2

- Set the initial pose (using the “2D Pose Estimate” tool).
- Send navigation goals (using the “Nav2 Goal” tool).

5. Tune and Test

- Adjust parameters in `nav2_params.yaml` for speed, safety, and responsiveness.
- Test in a controlled environment before deploying on water.

- Use logs and RViz to monitor performance.

Summary Table

Step	File/Command	Purpose
Configure Nav2 parameters	config/nav2_params.yaml	Navigation settings (controllers, costmaps)
Create launch file	launch/navigation.launch.py	Launch Nav2 stack
Launch navigation	ros2 launch ... navigation.launch.py	Start navigation system
Visualize & control	rviz2	Set pose, send goals, monitor
Tune/test	Edit YAML, observe in RViz	Optimize performance

HOW TO RUN AND USE YOUR URDF

- **Publish the URDF to the ROS 2 TF tree:**

Use the `robot_state_publisher` node, either directly or via your launch files.

- **Command-line example:**

```
ros2 run robot_state_publisher robot_state_publisher --ros-args -p
robot_description:="$(catamaran.urdf)"
```

- **Visualization:**

Open RViz2, add the "RobotModel" display, and you'll see your model and sensor frames.

- **Integration:**

Your launch files (`slam.launch.py` and `navigation.launch.py`) should already include the `robot_state_publisher` node, so running those will automatically publish your URDF.

```
catamaran.urdf
<?xml version="1.0"?>

<robot name="catamaran">

  <!-- Main bridge/deck -->
  <link name="base_link">
    <visual>
      <geometry>
        <box size="2.35 0.97 0.15"/>
      </geometry>
      <origin xyz="0 0 0.075"/>
      <material name="Aluminum">
        <color rgba="0.7 0.7 0.7 1.0"/>
      </material>
    </visual>
  </link>

  <!-- Starboard (right) hull -->
  <link name="starboard_hull">
    <visual>
      <geometry>
        <box size="2.35 0.20 0.30"/>
      </geometry>
    </visual>
  </link>
</robot>
```

```

    </geometry>
    <origin xyz="0 -0.385 0.15"/>
    <material name="Blue">
      <color rgba="0.1 0.1 0.8 1.0"/>
    </material>
  </visual>
</link>
<joint name="starboard_hull_joint" type="fixed">
  <parent link="base_link"/>
  <child link="starboard_hull"/>
  <origin xyz="0 -0.385 0"/>
</joint>

<!-- Port (left) hull -->
<link name="port_hull">
  <visual>
    <geometry>
      <box size="2.35 0.20 0.30"/>
    </geometry>
    <origin xyz="0 0.385 0.15"/>
    <material name="Red">
      <color rgba="0.8 0.1 0.1 1.0"/>
    </material>
  </visual>
</link>
<joint name="port_hull_joint" type="fixed">
  <parent link="base_link"/>
  <child link="port_hull"/>
  <origin xyz="0 0.385 0"/>
</joint>

<!-- GNSS (GPS) sensor (top, center) -->
<link name="gnss_link">
  <visual>
    <geometry>
      <cylinder length="0.05" radius="0.04"/>
    </geometry>
    <material name="White">
      <color rgba="1 1 1 1"/>
    </material>
  </visual>
</link>
<joint name="gnss_joint" type="fixed">
  <parent link="base_link"/>
  <child link="gnss_link"/>

```

```

    <origin xyz="0 0 0.40"/>
</joint>

<!-- RP Lidar (center of bridge) -->
<link name="rplidar_link">
  <visual>
    <geometry>
      <cylinder length="0.07" radius="0.06"/>
    </geometry>
    <material name="Black">
      <color rgba="0 0 0 1"/>
    </material>
  </visual>
</link>
<joint name="rplidar_joint" type="fixed">
  <parent link="base_link"/>
  <child link="rplidar_link"/>
  <origin xyz="0 0 0.18"/>
</joint>

<!-- Waterproof ultrasonic (front, center, just above waterline) -->
<link name="ultrasonic_front_link">
  <visual>
    <geometry>
      <cylinder length="0.03" radius="0.02"/>
    </geometry>
    <material name="Green">
      <color rgba="0.2 0.8 0.2 1"/>
    </material>
  </visual>
</link>
<joint name="ultrasonic_front_joint" type="fixed">
  <parent link="base_link"/>
  <child link="ultrasonic_front_link"/>
  <origin xyz="1.1 0 0.08"/>
</joint>

<!-- HC-SR04 (rear, starboard hull, protected) -->
<link name="ultrasonic_back_link">
  <visual>
    <geometry>
      <cylinder length="0.025" radius="0.01"/>
    </geometry>
    <material name="Yellow">
      <color rgba="0.9 0.9 0.2 1"/>
    </material>
  </visual>
</link>

```

```

    </material>
  </visual>
</link>
<joint name="ultrasonic_back_joint" type="fixed">
  <parent link="base_link"/>
  <child link="ultrasonic_back_link"/>
  <origin xyz="-1.1 -0.35 0.10"/>
</joint>

<!-- IMU (center of mass, near base_link) -->
<link name="imu_link">
  <visual>
    <geometry>
      <box size="0.05 0.05 0.02"/>
    </geometry>
    <material name="Orange">
      <color rgba="0.9 0.5 0.1 1"/>
    </material>
  </visual>
</link>
<joint name="imu_joint" type="fixed">
  <parent link="base_link"/>
  <child link="imu_link"/>
  <origin xyz="0 0 0.12"/>
</joint>

<!-- Starboard (right) rear thruster -->
<link name="thruster_starboard_link">
  <visual>
    <geometry>
      <cylinder length="0.10" radius="0.04"/>
    </geometry>
    <material name="Silver">
      <color rgba="0.7 0.7 0.7 1"/>
    </material>
  </visual>
</link>
<joint name="thruster_starboard_joint" type="fixed">
  <parent link="base_link"/>
  <child link="thruster_starboard_link"/>
  <origin xyz="-1.175 -0.385 0.05"/>
</joint>

<!-- Port (left) rear thruster -->
<link name="thruster_port_link">

```



```

<visual>
  <geometry>
    <cylinder length="0.10" radius="0.04"/>
  </geometry>
  <material name="Silver">
    <color rgba="0.7 0.7 0.7 1"/>
  </material>
</visual>
</link>
<joint name="thruster_port_joint" type="fixed">
  <parent link="base_link"/>
  <child link="thruster_port_link"/>
  <origin xyz="-1.175 0.385 0.05"/>
</joint>

</robot>

```

What About the Generated Files?

- **my_map.yaml**: Map metadata, generated when you save a map after SLAM.
- **my_map.pgm**: Occupancy grid image, generated with the map.
- **build/, install/, log/**: Created by colcon build for build artifacts, installed packages, and logs.

How to Get the Missing Files

navigate_to_pose_bt.xml and **navigate_through_poses_bt.xml**:

- Copy from the default Nav2 behavior trees, usually found in the **nav2_bt_navigator** package or online Nav2 resources.
- You can customize them later if your mission requires special behaviors.

navigate_to_pose_bt.xml

```

<root main_tree_to_execute="MainTree">
  <BehaviorTree ID="MainTree">
    <PipelineSequence name="NavigateToPose">
      <RecoveryNode number_of_retries="1">
        <SequenceStar name="navigation_recovery">
          <SetBlackboard output_key="goal" value="{goal}" />
          <ComputePathToPose goal="{goal}" path="{path}" planner_id="" />
          <FollowPath path="{path}" controller_id="" goal="{goal}" />
          <ClearCostmapExceptRegion map_subtype="local_costmap"
reset_distance="2.0"/>
        </SequenceStar>
      </RecoveryNode>
      <GoalReached/>
    </PipelineSequence>
  </BehaviorTree>
</root>

```

navigate_through_poses_bt.xml

```
<root main_tree_to_execute="MainTree">
  <BehaviorTree ID="MainTree">
    <PipelineSequence name="NavigateThroughPoses">
      <RecoveryNode number_of_retries="1">
        <SequenceStar name="navigation_recovery">
          <SetBlackboard output_key="goals" value="{goals}" />
          <ComputePathThroughPoses goals="{goals}" path="{path}" planner_id="" />
          <FollowPath path="{path}" controller_id="" goal="{goals[-1]}" />
          <ClearCostmapExceptRegion map_subtype="local_costmap"
reset_distance="2.0"/>
        </SequenceStar>
      </RecoveryNode>
    </PipelineSequence>
  </BehaviorTree>
</root>
```

package.xml:

- Generate with:

```
ros2 pkg create --build-type ament_python catamaran_navigation
```

or

```
ros2 pkg create --build-type ament_cmake catamaran_navigation
```

- Edit to include dependencies like nav2_bringup, slam_toolbox, robot_state_publisher, etc..

package.xml

```
<?xml version="1.0"?>
<package format="3">
  <name>catamaran_navigation</name>
  <version>0.1.0</version>
  <description>Navigation stack and configuration for the autonomous
catamaran.</description>
  <maintainer email="your.email@domain.com">Your Name</maintainer>
  <license>Apache-2.0</license>

  <buildtool_depend>ament_cmake</buildtool_depend>

  <!-- Navigation and SLAM -->
  <depend>navigation2</depend>
  <depend>nav2_bringup</depend>
  <depend>slam_toolbox</depend>

  <!-- Core robot and ROS 2 tools -->
  <depend>robot_state_publisher</depend>
  <depend>rviz2</depend>
  <depend>xacro</depend>
```

```
<depend>sensor_msgs</depend>
<depend>geometry_msgs</depend>
<depend>std_msgs</depend>
<depend>launch_ros</depend>

<!-- Nav2 components (explicit for clarity, some are included by navigation2) -
->
<depend>nav2_amcl</depend>
<depend>nav2_map_server</depend>
<depend>nav2_bt_navigator</depend>
<depend>nav2_controller</depend>
<depend>nav2_planner</depend>
<depend>nav2_lifecycle_manager</depend>

<export>
  <build_type>ament_cmake</build_type>
</export>
</package>
```